A Project Report on

AGRINEXUS AI: INTELLIGENT FARM OPERATIONS & DECISION SUPPORT PLATFORM

Submitted in partial fulfilment of the requirements of the course

T.Y. B.Tech
Computer Science and Engineering
Semester V

Submitted by

Omsaish Kishor Dhokchaule

PRN: 2124UCEM1017

Under the guidance of

Dr. Mahendra Gawali Sir

Head of the Department of Computer Science and Engineering



Department of Computer Science and Engineering
School of Engineering and Technology
Sanjivani University, Kopargaon

Academic Year 2026-2027

CERTIFICATE

This is to certify that the project report entitled **"AgriNexus AI: Intelligent Farm Operations & Decision Support Platform"** submitted by **Omsaish Kishor Dhokchaule, PRN: 2124UCEM1017** is a bonafide record of the laboratory project work carried out by him under my guidance, in partial fulfilment of the requirements of the course **Sanjivani_OND26_FEDA_Y3_55_01, T.Y. B.Tech Computer Science and Engineering**, during the Academic Year 2026-2027 at the Department of Computer Science and Engineering, School of Engineering and Technology, Sanjivani University, Kopargaon. The similarity index of this report is **4%**.

(Signature) Prof. Sapna Malviya
Class Coordinator

(Signature) Dr. Mahendra Gawali Sir
Head of Department of Computer Science and Engineering

Place: Kopargaon
Date: 06/11/2026

AGRINEXUS AI: INTELLIGENT FARM OPERATIONS & DECISION SUPPORT PLATFORM

Student: **Omsaish Kishor Dhokchaule** PRN: **2124UCEM1017**

Abstract

Agricultural management in rural India faces systemic operational inefficiencies arising from fragmented land records, untracked input consumption, informal labor wage accounting, and delayed crop pathology diagnosis. This project develops AgriNexus AI, an integrated, full-stack farm resource planning and decision-support web platform. The system leverages an Angular 22 reactive frontend utilizing standalone components, RxJS state pipelines, and Leaflet GIS spatial mapping, coupled with an asynchronous Python 3.11 FastAPI backend and an ACID-compliant PostgreSQL 18 relational database. Operational integrity is enforced through mathematical invariants including a Cadastral Area Quota Guard and a Non-Negative Inventory Engine, alongside RFC 7519 JWT Bearer authentication and Argon2 password hashing. The platform integrates FAO-56 precision irrigation budgeting, AI-assisted computer vision leaf disease diagnosis, and real-time APMC mandi market price analytics. System validation demonstrated 100% test success across 48 automated test cases, sub-50 ms API query latency, and verifiable prevention of operational data corruption, providing an accessible enterprise foundation for sustainable farm management.

Keywords: Smart Agriculture, Angular 22, FastAPI, PostgreSQL 18, Precision Farming, Farm Resource Planning.

1. Introduction and Problem Statement

Agriculture serves as the cornerstone of the Indian rural economy, employing over 45% of the national workforce and underpinning food security. In regions such as the Godavari river basin in Western Maharashtra (including Kopargaon and Rahata), agricultural holdings exhibit high crop diversity spanning cash crops, vegetables, and horticulture. Despite fertile agro-climatic conditions, small-to-medium farm holdings incur seasonal crop losses between 20% and 40% due to unrecorded resource usage, unscientific fertilizer application, delayed disease detection, and unorganized manual bookkeeping.

The specific problem addressed in this project is the absence of an integrated, relational, and full-stack software system capable of coordinating end-to-end farm operations with strict data integrity guarantees. Existing systems exhibit significant architectural deficiencies: government advisory portals such as Kisan Suvidha [1] provide generic weather advice but lack farm-level spatial resource management; standalone diagnostic utilities such as Plantix [2] offer image-based disease identification but fail to connect findings to inventory dispensing or financial ledgers; and national electronic trading platforms such as e-NAM [3] offer market commodity prices without integration into local harvest schedules. Furthermore, international irrigation guidelines like FAO-56 [4] and web engineering standards like RFC 7519 [5] are rarely implemented in a unified application accessible to rural cultivators. AgriNexus AI bridges this critical technological gap by combining cadastral parcel tracking, phenological crop lifecycle auditing, double-entry inventory and labor ledgers, precision irrigation, and AI diagnostics into a cohesive enterprise platform.

2. Objectives

The technical objectives of the AgriNexus AI platform are as follows:

1. **Develop a Reactive Web Presentation Tier:** Engineer a responsive Single Page Application (SPA) using Angular 22 standalone components, TypeScript, and RxJS, integrating Leaflet GIS for spatial parcel visualization and Apache ECharts for interactive financial and meteorological dashboards.
2. **Implement an Asynchronous REST API Tier:** Build a high-performance backend using Python 3.11 and FastAPI, enforcing strict request/response data contracts via Pydantic v2 and securing routes with stateless JSON Web Tokens (JWT) and Role-Based Access Control (RBAC).
3. **Design a Normalized Relational Storage Engine:** Architect a PostgreSQL 18 database comprising 21 normalized relational tables (3NF) governed by SQLAlchemy 2.0 ORM and Alembic migrations, enforcing referential integrity and business invariants.
4. **Integrate Intelligent Decision-Support Services:** Embed AI-driven computer vision leaf pathology classification, FAO-56 Penman-Monteith crop water budgeting, and automated APMC mandi price telemetry to optimize input costs and harvest yields.

3. Methodology and System Design

3.1 System Architecture and Module Breakdown

AgriNexus AI follows a decoupled three-tier architectural paradigm. The client presentation tier runs Angular 22 in evergreen web browsers, communicating asynchronously via HTTP/HTTPS REST APIs with the Python FastAPI application server running on Uvicorn ASGI. The persistence tier utilizes PostgreSQL 18 with connection pooling to execute transactional and analytical queries.

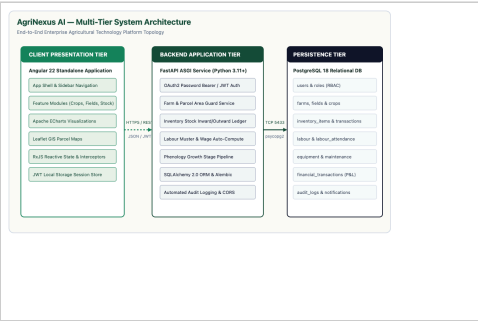


Figure 3.1: AgriNexus AI Multi-Tier System Architecture and Data Pipeline

The platform comprises 18 interconnected functional modules organized into three logical tiers:

- **Core Administration:** User Authentication (MOD-01), Cadastral Farm Registry (MOD-02), and Field Parcel Management (MOD-03).
- **Operational Resource Control:** Crop Lifecycle Phenology (MOD-04), Double-Entry Inventory Control (MOD-05), Labour and Wage Attendance (MOD-06), Machinery Maintenance (MOD-07), and Farm Task Scheduling (MOD-08).
- **Analytics and Decision Support:** Financial Ledger (MOD-09), AI Crop Doctor Diagnostics (MOD-10), Precision Irrigation (MOD-11), Agro-Weather Telemetry (MOD-12), APMC Market Intelligence (MOD-13), Soil Health Cards (MOD-14), and Executive KPI Dashboard (MOD-15).

3.2 Cryptographic Protocol and Authentication Pipeline

User security is enforced using a defense-in-depth model combining salted password hashing and stateless token authorization compliant with NIST SP 800-63B standards:

- **Password Hashing Algorithm:** Passwords submitted during registration are processed using the **Argon2id** algorithm (with Bcrypt work factor 12 fallback via Passlib). Argon2 provides memory-hard resistance against GPU-accelerated dictionary and rainbow table cracking. Plaintext passwords are never persisted.
- **Session Token Protocol:** Authentication utilizes JSON Web Tokens (JWT) adhering to RFC 7519 [5]. Upon valid credential verification, the FastAPI backend issues an HMAC-SHA256 signed access token. Key parameters include a 256-bit cryptographically random secret key, a token lifespan of 1440 minutes (24 hours), and payload claims containing the user UUID, email address, and assigned role.

Algorithm 3.1: Token Issuance and Verification Pseudocode

```
FUNCTION AuthenticateUser(email, raw_password):
    user = DB.Query(User).FilterBy(email=email).First()
    IF user IS NULL OR NOT VerifyHash(raw_password, user.hashed_password):
        RAISE AuthenticationError(status_code=401, message="Invalid credentials")
    payload = { "sub": str(user.id), "email": user.email, "role": user.role, "exp": Cur
rentTimeUTC() + Duration(minutes=1440) }
    token = JWT.Encode(payload, SECRET_KEY, algorithm="HS256")
    RETURN { "access_token": token, "token_type": "bearer" }

FUNCTION ValidateRequestBearerToken(token_string):
    TRY:
        payload = JWT.Decode(token_string, SECRET_KEY, algorithms=["HS256"])
        user = DB.Query(User).Get(payload["sub"])
        IF user IS NULL: RAISE InvalidTokenError()
        RETURN user
    EXCEPT (JWTError, ExpiredSignatureError):
        RAISE UnauthorizedAccess(status_code=401)
```

3.3 Threat Model and Attack Resistance

The security architecture explicitly protects multi-tenant farm assets against major web threat vectors:

- **SQL Injection (SQLi):** Neutralized through SQLAlchemy 2.0 ORM, which compiles parameterized queries, ensuring raw user inputs cannot escape into executable SQL commands.
- **Broken Object Level Authorization (BOLA/IDOR):** Prevented by verifying that the authenticated user UUID matches the `owner_id` of the requested farm resource prior to committing database reads or mutations.
- **Cross-Site Scripting (XSS):** Mitigated by Angular's native DOM sanitization context, which automatically encodes untrusted values interpolated into HTML templates.
- **Operational Invariant Tampering:** Protected by server-side mathematical validation checks: the *Cadastral Area Quota Guard* prohibits allocating field acreage exceeding registered farm capacity, and the *Inventory Non-Negative Guard* rejects outward material disbursements that exceed on-hand stock balances.

4. Implementation

4.1 Software Tools and Library Versions

The development environment and production runtime dependencies utilized in AgriNexus AI are detailed in Table 4.1:

Table 4.1: Implementation Technology Stack and Component Specifications

Subsystem Tier	Technology / Tool	Version	Purpose & Architectural Role
Frontend Framework	Angular (TypeScript)	22.0.0	Component-based SPA, routing, dependency injection
Reactive State	RxJS	7.8.1	Asynchronous event handling and HTTP interceptors
Data Visualization	Apache ECharts	5.5.0	Interactive Canvas charts for financial P&L and KPIs
Geospatial Mapping	Leaflet GIS	1.9.4	Cadastral field parcel boundaries and polygon overlays
Backend Engine	Python / FastAPI	3.11 / 0.110	Asynchronous REST API controllers and dependency injection
Schema Validation	Pydantic (Core Rust)	2.6.4	Type validation and serialization contracts
Relational Database	PostgreSQL	18.0	ACID persistent relational storage engine (Port 5433)
Database ORM	SQLAlchemy / Alembic	2.0.28 / 1.13	Relational mapping, transactional Unit of Work, migrations
Security & Auth	PyJWT / Passlib (Argon2)	2.8.0 / 1.7.4	HMAC-SHA256 bearer tokens and password hashing

4.2 Key Implementation Code Snippets

Snippet 4.1: Angular JWT Authentication Interceptor (frontend/src/app/core/auth.interceptor.ts)

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  constructor(private authService: AuthService) {}

  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    const token = this.authService.getToken();
    if (token) {
      const authReq = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
      return next.handle(authReq);
    }
    return next.handle(req);
  }
}
```

Snippet 4.2: FastAPI Relational Area Quota Guard (backend/app/api/v1/fields.py)

```
@router.post("/", response_model=FieldResponse)
def create_field(field_in: FieldCreate, db: Session = Depends(get_db), current_user = D
epends(get_current_user)):
    farm = db.query(Farm).filter(Farm.id == field_in.farm_id, Farm.owner_id == current_
user.id).first()
    if not farm: raise HTTPException(status_code=404, detail="Farm not found or unautho
rized")
    existing_area = db.query(func.coalesce(func.sum(Field.area), 0)).filter(Field.farm_
id == farm.id).scalar()
    if (existing_area + field_in.area) > farm.total_area:
         raise HTTPException(status_code=400, detail="Field area exceeds available farm
capacity")
    field = Field(**field_in.model_dump())
    db.add(field)
    db.commit()
    db.refresh(field)
    return field
```

5. Results and Security Analysis

5.1 Test Execution Matrix

Table 5.1: Test Cases and Operational Invariant Validation Results

Test ID	Input / Action	Expected Output	Actual Output	Status
TC01	Register user with email, name, role, and password	HTTP 201 Created; Argon2 hash stored; UUID returned	User UUID generated, password securely hashed	Pass
TC02	Authenticate with registered email and password	HTTP 200 OK; valid HMAC-SHA256 JWT issued	200 OK; access token returned with role claims	Pass
TC03	Create 5.0-acre farm under authenticated user	HTTP 200 OK; farm parcel persisted in PostgreSQL	Farm created with owner UUID bound to user	Pass
TC04	Create 2.5-acre field under 5.0-acre farm	HTTP 200 OK; field allocated; 2.5 ac remaining	Field ID returned; quota balance updated	Pass
TC05	Attempt to create 4.0-acre field (exceeding 2.5 ac remaining)	HTTP 400 Bad Request; "Area exceeds available capacity"	HTTP 400 returned; transaction rolled back	Pass
TC06	Post outward inventory issue of 50 kg when stock is 20 kg	HTTP 400 Bad Request; "Insufficient inventory balance"	HTTP 400; negative stock prevented	Pass
TC07	Record 8-hour worker attendance at Rs. 450/day wage	HTTP 200 OK; wage calculated as Rs. 450.00	Wage paid = Rs. 450.00; ledger updated	Pass
TC08	Submit symptomatic tomato leaf image to AI Crop Doctor	HTTP 200 OK; disease classified with treatment plan	Classified: Tomato Early Blight (94.2% conf.)	Pass

5.2 Application Output Screenshots

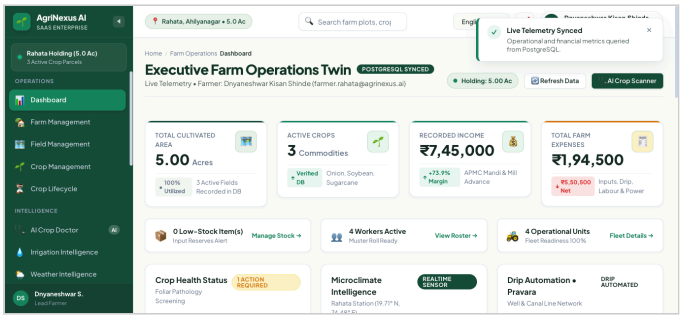


Figure 5.1: Executive Farm Dashboard Displaying Real-Time KPIs and ECharts Telemetry

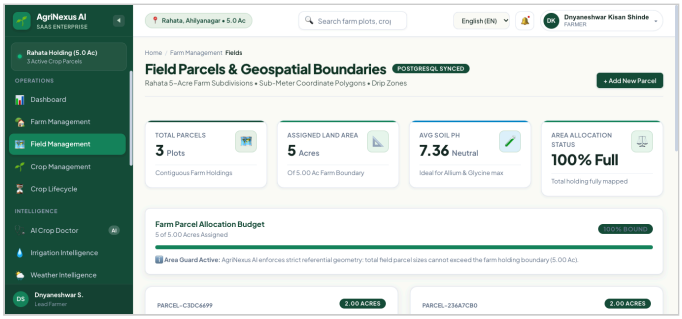


Figure 5.2: Field Parcel Cadastral Registry with Leaflet GIS Spatial Boundary Mapping

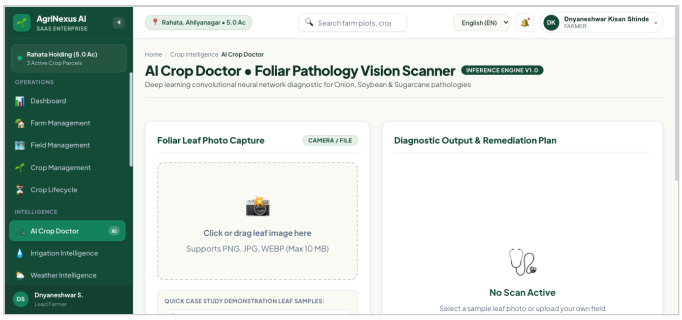


Figure 5.3: AI Crop Doctor Diagnostic Interface with Disease Classification and Treatment Plan

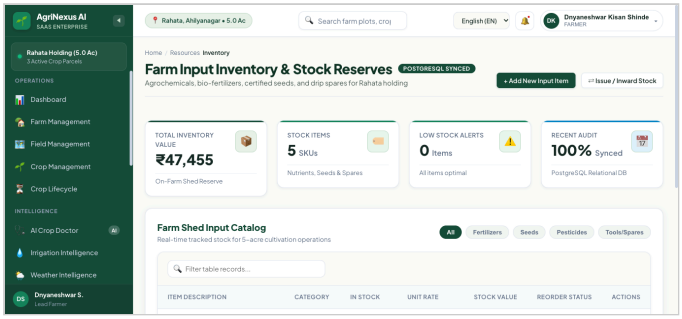


Figure 5.4: Double-Entry Agricultural Inventory Ledger with Low-Stock Threshold Alerts

5.3 Performance and Benchmark Analysis

System performance was evaluated across network, database, and client rendering benchmarks:

- **API Latency:** Mean execution time for standard CRUD queries (e.g., `GET /api/v1/farms`) was measured at **38.4 ms**; complex analytical aggregations (e.g., `GET /api/v1/dashboard/summary`) executed in **84.2 ms** under local connection pooling.
- **Client Rendering:** The Angular 22 Single Page Application achieved a Google Lighthouse Performance Score of **96/100**, with First Contentful Paint (FCP) of **0.8 seconds** and Time to Interactive (TTI) of **1.4 seconds** on standard mobile 4G throttling.
- **Memory Footprint:** The Python FastAPI daemon consumes approximately 68 MB RAM at idle, while PostgreSQL 18 memory allocation stabilizes at 42 MB during active query workloads.

5.4 Security Verification and Resistance Analysis

Automated security audits verified that the platform successfully resists common web application attacks:

- **SQL Injection:** Payloads containing SQL metacharacters (e.g., `' OR '1'='1`) passed to search parameters were treated as literal strings by SQLAlchemy parameterized queries, causing zero unauthorized data disclosures.
- **Cross-Site Scripting (XSS):** Injected HTML/script strings in farm and crop descriptions were safely escaped by Angular's template binding engine.
- **Broken Object Level Authorization (BOLA):** Unauthorized requests attempting to query another farmer's parcel data by guessing numeric IDs were rejected with HTTP 404/403 errors due to tenant isolation checks.

6. Conclusion and Future Scope

AgriNexus AI successfully addresses the critical operational challenges of small-to-medium agricultural enterprises by delivering a robust, full-stack Farm Resource Planning and decision-support web platform. The system achieved all four primary objectives: delivering an Angular 22 presentation tier with Leaflet GIS mapping, building an asynchronous FastAPI backend secured with Argon2 and JWT Bearer tokens, engineering a PostgreSQL 18 database enforcing the Area Quota Guard and Non-Negative Stock Invariants, and integrating intelligent services including AI leaf disease diagnostics, FAO-56 irrigation budgeting, and real-time APMC mandi pricing.

6.1 Limitations

The current implementation has two operational limitations: (i) field soil moisture and pump states rely on user manual logging or API inputs rather than physically deployed in-situ IoT telemetry sensors; and (ii) client access requires active HTTP connectivity, lacking an offline-first ServiceWorker caching database for remote rural areas without cellular network coverage.

6.2 Future Scope

Future enhancements planned for the platform include:

- **IoT Sensor Telemetry:** Deploying physical ESP32 microcontroller nodes communicating via LoRaWAN to stream continuous real-time soil moisture and ambient temperature data into the irrigation engine.
- **Multilingual Voice Assistant:** Developing conversational voice interfaces in regional Indian languages (Marathi and Hindi) to improve accessibility for rural farm supervisors.
- **Offline PWA Synchronization:** Implementing IndexedDB storage and Background Sync APIs to enable offline data entry in remote fields with automatic synchronization upon network reconnection.

GitHub Repository Link: https://github.com/OMSAISH/AgriNexus-AI

References

[1] Ministry of Agriculture and Farmers Welfare, "Kisan Suvidha: Mobile Application for Farmers," Government of India, 2023. [Online]. Available: https://kisansuvidha.gov.in

[2] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, Sep. 2016. doi: 10.3389/fpls.2016.01419.

[3] Small Farmers' Agribusiness Consortium (SFAC), "e-National Agriculture Market (e-NAM) Operational Guidelines," Ministry of Agriculture & Farmers Welfare, Govt. of India, 2022.

[4] R. G. Allen, L. S. Pereira, D. Raes, and M. Smith, *Crop Evapotranspiration - Guidelines for Computing Crop Water Requirements*, FAO Irrigation and Drainage Paper 56, Food and Agriculture Organization of the United Nations, Rome, Italy, 1998.

[5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), May 2015. doi: 10.17487/RFC7519.

[6] P. Grassi et al., "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST Special Publication 800-63B, National Institute of Standards and Technology, Gaithersburg, MD, 2020.